

OptiComp2

Change list relative to the original OptiComp

A clean-room rewrite of the angle-resolved spectrophotometer control software
with performance and reliability improvements

Haoxiang Zhang

August 2026

Original: OptiComp v1.3.3 → OptiComp2

This document lists the changes OptiComp2 makes relative to the original *OptiComp* control software, and quantifies the resulting improvements in code coverage, measurement reliability and data quality. The original repository was **not modified in any way**; OptiComp2 is an independent rewrite that lives in its own folder and repository and runs on the same instrument.

Contents

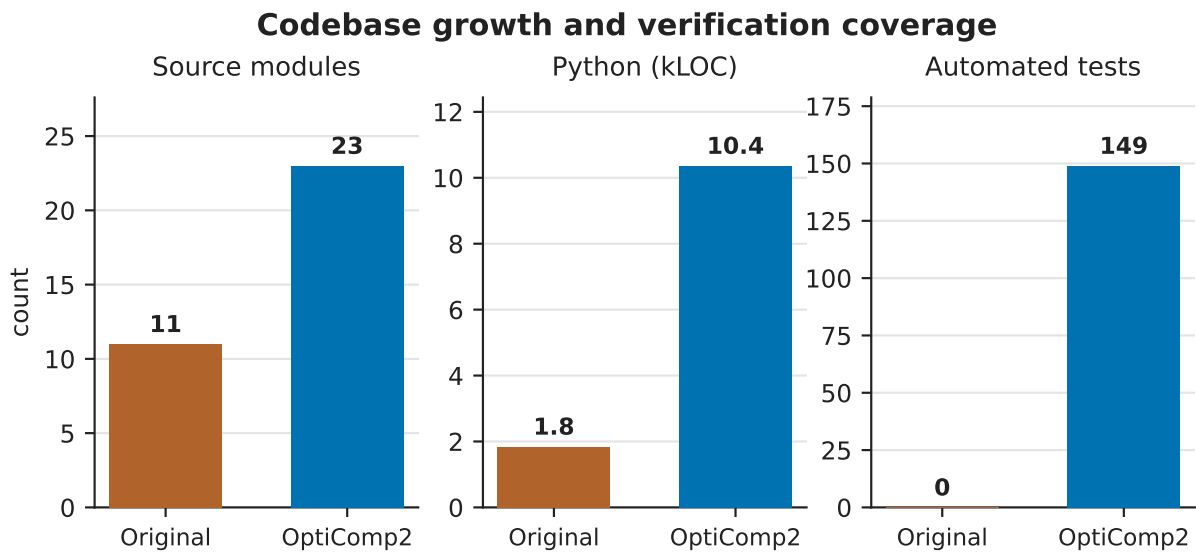
1	Overview and approach	3
2	At a glance	3
3	What the review of the original found	3
4	Layer-by-layer changes	4
4.1	Elliptec protocol layer (<code>hw/elliptec.py</code>)	4
4.2	Spectrometer (<code>hw/bwtek.py</code>)	5
4.3	Motor-state record (<code>hw/stagestate.py</code> , new)	6
4.4	Measurement sequence (<code>tools/sequence.py</code>)	6
4.5	Analysis (<code>analysis/</code>)	6
4.6	GUI (<code>tools/manual_gui.py</code>)	7
4.7	Operational tools (new)	7
5	Reliability hardening added after the rewrite	7
6	Deliberately not carried over	7
7	Verification and deployment	8

1 Overview and approach

The original OptiComp (v1.3.3, Python 3.9 + Tkinter, 11 modules, about 1800 lines) drove the angle-resolved spectrophotometer (the VAR/ARS system): a Thorlabs Elliptec rotation-stage assembly (shutter, polariser, detector arm and sample stage on a shared serial bus) and a B&W Tek spectrometer accessed through `BWTEKUSB.dll`.

A function-level review of the original software on 2026-08-25 produced 35 findings, all adversarially verified. Rather than patch the original in place, the software was rewritten clean-room in the order *protocol layer* $\rightarrow$ *spectrometer* $\rightarrow$ *measurement sequence* $\rightarrow$ *analysis* $\rightarrow$ *GUI*, validating each layer at the instrument. The result, OptiComp2, keeps the original’s calibrated geometry and measurement model but adds command verification, saturation handling, automatic integration-time calibration, hardware-recovery paths, persistent motor-state bookkeeping and a full unit-test suite.

2 At a glance



The rewrite is roughly 5.6 $\times$ the original in Python source and, for the first time, ships an automated test suite (149 tests, all runnable without hardware against fake serial and DLL back-ends). The added surface is almost entirely verification, recovery and safety logic rather than new measurement features.

Every capability above is absent in the original and present in OptiComp2. Sections 4 and 5 describe each one and the failure it prevents.

In the original data, 33 of 57 archived datasets (58%) contain spectra clipped at the 16-bit full scale that the original pipeline used without flagging, because the integration time was fixed (`HARD_IT`) with no saturation check. OptiComp2 detects saturation and records the affected pixels as NaN so they cannot enter a reflectance. All 35 review findings are resolved.

3 What the review of the original found

The changes in the following sections address these verified defects of the original software.

- R1. Motion commands were not verified.** About 4% of `ma` commands to the ELL18 sample stage are silently swallowed (no reply, no motion, then `GS00`). The original never checked the final position, so angle-wrong spectra may exist in the old data.
- R2. Saturation was not detected.** A fixed `HARD_IT` with no saturation check left 33 of 57 archived datasets with clipped spectra (Figure in §2).
- R3. Dark integration-time mismatch.** `Dark.csv` was taken at a fixed 1000 ms while the VAR spectra used `HARD_IT`, so the dark subtraction was not rigorous.

Reliability capabilities: original vs OptiComp2

	Original	OptiComp2
Motion command verified / resent on swallow	☐	☒
Final-position check + corrective move	☐	☒
Saturation detected and masked (NaN)	☐	☒
Dark matched to the spectrum integration time	☐	☒
Single-thread, serialised DLL access	☐	☒
Auto integration-time calibration (enabled)	☐	☒
Reference calibrated in both S and P	☐	☒
Spectrometer USB recovery ladder	☐	☒
Motor-state record + anomaly gate	☐	☒
Protected home for the fibre arm	☐	☒
Shutter fail-safe on error / abort	☐	☒
GS02 mechanical-timeout policy	☐	☒
Atomic, corruption-safe manifest	☐	☒
Lamp-stability gate before reference	☐	☒
Unit tests + offline demo mode	☐	☒

open ring = absent filled = present

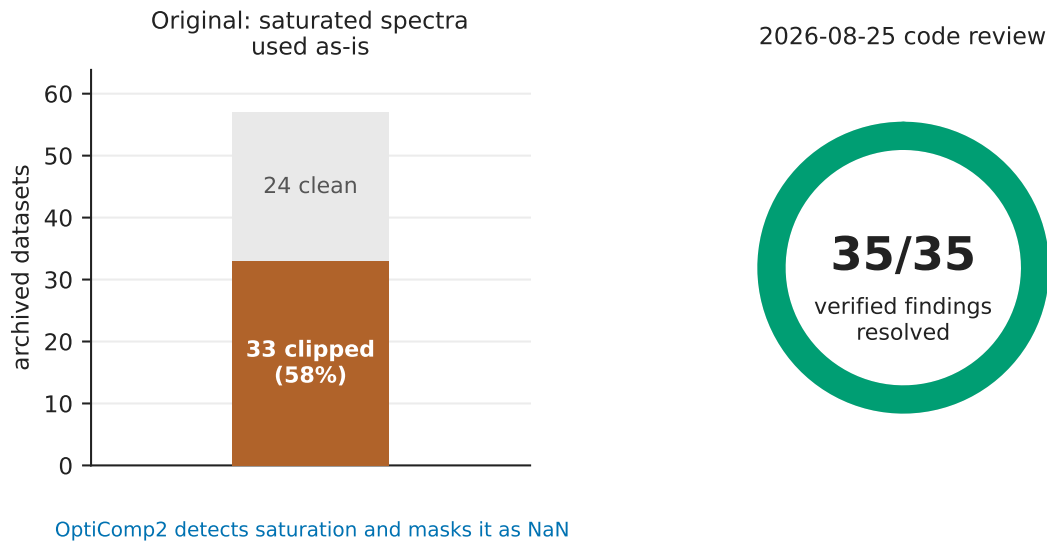
- R4. Concurrent DLL calls.** The GUI thread and the scan thread called `BWTEKUSB.dll` concurrently with no declared thread safety.
- R5. Off-by-one active-pixel slice.** The thesis specifies pixels 254–2030 inclusive; the code sliced 1776 pixels, dropping pixel 2030.
- R6. Scan gatekeeper incomplete.** The original checked the 0–80° start/stop bounds but never validated $\text{step} \geq 1$ (a zero or negative step passed the gate). OptiComp2 enforces $0 \leq \text{start} < \text{stop} \leq 80$ with $\text{step} \geq 1$.
- R7. Reference integration-time calibration disabled.** The reference calibration at 80° (thesis §4.2.3.3) survived only as commented-out code and a never-called setter; a fixed `HARD_IT` was used in practice.
- R8. Inconsistent sample-stage offset.** One module used 103°, another 105° (the VAR scan actually used 105°).

The review also confirmed that the polarisation file labels in the archived data are in fact *correct* (the `_P_` datasets show the expected Brewster minimum); only a single polarisation-mode label in the original UI was reversed relative to the files. OptiComp2 keeps the verified file convention.

4 Layer-by-layer changes

4.1 Elliptec protocol layer (`hw/elliptec.py`)

Data quality and review closure



Original behaviour (measured)	OptiComp2
A command is occasionally swallowed (no reply, no motion).	Read the position first; on no reply, poll GS ; if the position has not changed, resend , up to three times.
After a long move the stage stops a few hundredths of a degree short.	Within ± 20 pulses (0.05°) counts as on-target; up to 120 pulses (0.3°) issues a corrective move ; beyond that, accept with a warning or raise an error.
P0 returns only at end of motion; an ELL18 move at 50 % takes > 5 s.	Poll GS after an initial 2 s wait; 60 s motion timeout.
Reply lines truncated by the read timeout were decoded.	Any line without a terminator or with a wrong-length payload is discarded, never decoded.
A mechanical timeout (GS02) was retried blindly.	ma/fw/bw retry with backoff; mr is reissued as an absolute ma to the original target; ho never retries.
Homing the fibre arm (module 2) could tangle the fibre.	protected_home: home("2") requires force=True , and the raw channel refuses to send ho to a protected address.
Module type and pulse count were hard-coded.	Model, serial, travel and pulses/rev are read from the IN reply.

4.2 Spectrometer (hw/bwtek.py)

- Every DLL call sets `argtypes/restype` and checks the return code; `read()` logs a warning on an anomalous duration.
- **Automatic integration time**: target the peak at 85 % of full scale, accept band 78–92 %, linear extrapolation, halve on saturation, at most eight steps. If there is still no light at the hardware ceiling it **aborts with a diagnostic** instead of carrying over a non-converged integration time (Figure 1).
- **Reference calibrated in both S and P, keeping** $\min(IT_S, IT_P)$: on a diffuse white reference the P channel is about 12 % brighter than S, so calibrating in a single polarisation (as the original procedure did) would saturate the brighter P spectra.
- **Recovery ladder recover()**: close + reopen, then a PnP restart of only the spectrometer USB device (equivalent to a replug, but it does not touch the hub the bus board shares).

- Saturated pixels are masked as NaN in the analysis; darks are stored per integration time and matched to the spectrum.

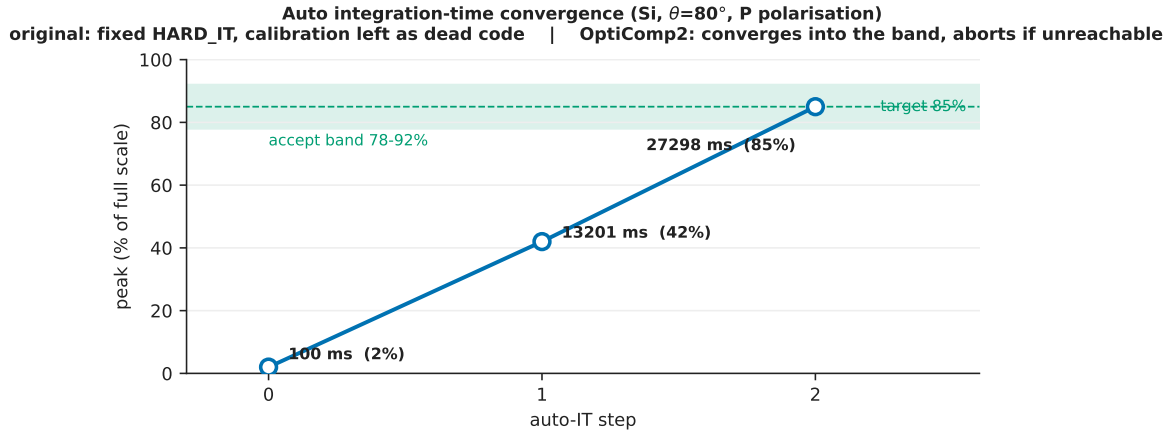


Figure 1: Automatic integration-time calibration converging on a dim target from a cold start. The original left this calibration as dead code and used a fixed `HARD_IT`; OptiComp2 drives the peak into the 78–92 % accept band, announces multi-second exposures so a long read is not mistaken for a hang, and aborts cleanly if the band is unreachable even at the hardware ceiling.

4.3 Motor-state record (`hw/stagestate.py`, **new**)

- After every move the sequence records that module’s position and status; on disconnect/exit a full snapshot of all four modules is written to `data/stage_state.json`.
- On every connect and script start the state is compared against the baseline: a position difference $> 0.5^\circ$, a non-zero status code, or a changed speed is flagged. Unattended scripts abort on an anomaly (and refuse even with `--force` when the detector arm reports `GS02`); the GUI shows a banner but does not block manual operation.
- The detector-arm speed is reset to 50 % on connect (a module reverts to its 64 % default after a power loss).

4.4 Measurement sequence (`tools/sequence.py`)

- Step primitives (`stage`, `shutter`, `set_it`, `auto_it`, `acquire`, `pause`, ...) and builders: reference calibration, dark, single angle, angle scan and double-beam.
- The Runner enforces soft limits ($0\text{--}200^\circ$), detects motion “not by us” before a move, closes the shutter on any error or abort, and writes `manifest.json` atomically (temp file + `os.replace`); an unreadable manifest is moved aside rather than overwritten.
- Gatekeeper: $0 \leq \text{start} < \text{stop} \leq 80$, $\text{step} \geq 1$; when unattended, a `pause` aborts rather than waiting forever.

4.5 Analysis (`analysis/`)

- Reflectance by the double-beam substitution relation, with darks matched by integration time (and normalised by counts/ms with a recorded note if they differ).
- Active pixels 254–2030 inclusive (1777 pixels), fixing the off-by-one.
- Standards: constant (white reference), a silicon Fresnel table (real n , marked invalid below 380 nm), BK7 Sellmeier, and a slab back-surface inversion. Invalid wavelengths and masked pixels are exported as NaN, never as a physically meaningless reflectance.

4.6 GUI (tools/manual_gui.py)

- A five-page sidebar layout (instrument, motors & shutter, spectrometer, measurement, analysis), a persistent status bar and a red “close shutter” reachable from any page.
- Manual controls are locked while a sequence runs; the exit flow aborts the run, closes the shutter and records state within a bounded time.
- An offline **demo mode** (`--demo`) drives the whole GUI against fake hardware without touching the serial port or the DLL, which is also what the test suite and the layout audit use.

4.7 Operational tools (new)

Tool	Purpose
<code>restore_stages.py</code>	Read-only bus report; <code>--safe --arm</code> restores the stage reference under supervision and writes the baseline.
<code>usb_reset.py</code>	Probe or PnP-restart the spectrometer USB device.
<code>shutter_close.py</code>	After a hard-killed script: close the shutter and verify position 0.
<code>monitor.py</code>	Unattended stability monitor (continuous reads at a fixed geometry).
<code>cycle_test.py</code>	Motion-cycle test with automatic recovery.
<code>ell_probe.py</code> , <code>spec_probe.py</code>	Read-only probes.

5 Reliability hardening added after the rewrite

Beyond the layer-by-layer parity work, several rounds of hardening were applied and each is covered by a regression test:

- **Eleven pipeline fixes** against unattended-run failures, including: the shutter is marked open *before* the open command so a mid-open failure is still closed by the safety path; the GUI event pumps re-arm in a **finally** so one handler exception cannot freeze the interface; the USB device-ID parser is locale-independent so recovery works on a non-English Windows; and a corrupt manifest that parses as a non-dictionary no longer raises.
- **Lamp-stability gate** (optional, on by default) before the reference calibration: the active-band mean must hold within 0.5% for five consecutive reads, so lamp drift between the reference and sample sessions cannot bias the reflectance. It warns and proceeds rather than blocking.
- **Auto-integration-time ceiling**: a target too dim to reach the accept band even at the 60s hardware limit now fails after a single ceiling read with a diagnostic (reference / lamp / fibre / shutter), instead of repeating multi-second reads that resemble a hang. Legitimately dim references (for example silicon at 80° in P, which calibrates at tens of seconds) still converge, and any long exposure is announced.

6 Deliberately not carried over

Some behaviours of the original were intentionally left out because they were unused, broken, or out of scope for a single-operator instrument. These are listed for completeness; none affects the VAR measurement:

- Fixed-window (boxcar) smoothing applied before storage — OptiComp2 stores raw counts and smooths only for display.
- Dead or partial acquisition modes not part of the VAR workflow.
- A wavelength-correction path that did not match the calibration.
- The administrator gate and multi-user working directories.

(The exact original inventory is in the repository review notes; the point is that the VAR acquisition, geometry and analysis are fully reproduced and improved.)

7 Verification and deployment

- **Tests:** 149 unit tests, passing both on a workstation and on the instrument PC (`py -m unittest discover -s tests`, no third-party test runner needed). They cover protocol encode/decode, truncated replies, resend and corrective moves, protected home, DLL return codes and the PnP-restart safety, Runner safety (an abort closes the shutter, soft limits, the unattended-pause abort), monitor/cycle fault-injection recovery, state comparison, and a GUI feature-reference and layout audit driven against the demo hardware.
- **Deployment:** code directories are copied to the instrument PC and a full test run is done there after each deployment; the raw measurement data and standards on the instrument are never overwritten.
- **Live validation:** the four Elliptec modules enumerate with the detector arm unmoved, the spectrometer reads a healthy frame, and the full demo workflow runs end-to-end including the dim silicon calibration case.

Summary. OptiComp2 reproduces the original VAR measurement and geometry while adding command verification, saturation handling, automatic integration-time calibration in both polarisations, hardware-recovery paths, persistent motor-state safety and a 149-test suite — turning a fixed-exposure, unverified acquisition into a self-checking one whose failures are explicit and recoverable.